

The empty blocks were the pause enforcing itself, and the flag day should move one day, not indefinitely

Reply to bubu's letter of 19 August ("testnet is not including any transactions"). Everything below was verified on the committee host today, with heights and identifiers given so each claim can be re-checked.

Your three questions, answered first

- 1. What build are the producers running?** All 20 committee producers and the dexnode run one uniform binary built 2026-08-16 from master commit `5e6a302d8`, reporting version 24.1.0. That commit is 41 commits past the `v24.1.0` tag, so it is a master snapshot, identical everywhere; the explorer-facing node runs the `v24.2.0` tag build. The mempool and supervision code is byte-identical between the two (`git diff 5e6a302d8 v24.2.0 -- src/txmempool.cpp src/validation.cpp src/supervision.cpp` is empty). The scheduled 24.3.0 cutover will put the whole set on a tagged release.
- 2. Do their mempools still hold a spend the pause invalidated?** No. At 16:34Z every mempool on the host was empty: node000, node001, node005, node010, node015, node019, the dexnode, and the explorer node all reported `size: 0`. There was no resident spend of `d1bd783c...` anywhere, and there still is none.
- 3. Do their logs mention Supervision: evicting ... around 10:13–10:15Z?** No, on any node, at any point today. Nothing was resident to evict. The logs do show the pause registering cleanly at 10:13:50Z: `Supervision: d1bd783c... freezes script 0000...0000`.

What actually happened

Your correlation is real and exact, and the causation runs the other way: the pause did not break the producers, it invalidated the one flow that was feeding them.

The evidence chain:

- **Block 99878 (10:12:44Z)** carries a transfer of asset `d1bd783c...`. **Block 99879 (10:13:44Z)** carries the pause record for that same asset (`SEQFRZ`, target `0000...0000`). This asset is yours, and the pause was an issuer action from your side.
- **Every peer link from this host to your node (13.140.162.77) shows the same frozen timestamp:** `last_transaction = 10:13:04Z`, unchanged for the following six hours. That field updates only when a *valid* transaction arrives. Your node kept live, relaying connections the whole time (four links, `relaytxes: true`, normal `minfeefilter`); what stopped at 10:13Z was

your transactions surviving validation here. From the pause onward, everything your node offered was a spend of the paused asset or a child chained onto one, and each was rejected as a frozen-asset spend or as an orphan of a rejected parent.

- **The rejections were invisible on both sides**, which is why this took hours to see: rejection logging lives under the `mempoolrej` debug category and is off by default. It is now enabled on the explorer node, so from 16:51Z onward every rejection at the public entry is logged with its reason.
- **Nothing on our host was submitting either.** In the same window the box services happened to be idle; the only submissions were wallet re-broadcasts of transactions that were already invalid. So for six hours the network was offered nothing valid, and the producers, correctly, built empty blocks on schedule. Your own observation confirms the producers were healthy: 382 blocks, every one on time. The failure mode you quoted from `feature_supervised_assets.py` point 9 predicts the opposite signature, producers skipping their slots. That never happened.

The window ended the moment anyone offered a valid transaction. The first fresh submission (16:36:25Z) confirmed at height 100262, twenty seconds later, followed within minutes by a faucet payout (100266) and staking transactions (100274 through 100280).

Inclusion verified end to end, today

Four independent paths, all confirming in one to three blocks:

entry point	probe	confirmed at
local RPC on a producer	244b23ee ...	100262
public HTTP (explorer <code>sendrawtransaction</code>)	bb417817 ...	100274
second RPC probe under full debug logging	5f8d450a ...	100277
pure P2P from a fresh external node, synced from genesis over the public port, exactly your topology (one 24.1.0 peer, one 24.2.0 peer)	3770559b ...	100306

What to check on your node

Your mempool retains transactions this network rejects, and a day of wallet activity has likely chained on top of them. Three steps separate the recoverable from the dead:

1. **Classify.** For each stuck txid, decode it and check every input's prevout asset. Anything touching `d1bd783c...` is dead until you unpause: the pause is still active, so those spends can

never confirm anywhere. Abandon them (`abandontransaction`), or lift the pause through your issuer flow if the test is complete.

2. **Rebroadcast the remainder.** A transaction is announced once per peer and never again on that link, and wallet resend timers run on the scale of hours. Push each still-valid transaction through the public entry (`sendrawtransaction` against the explorer) and you will now get either a confirmation or a logged, quotable rejection reason.
3. **Tell us your node's build.** You asked for ours; yours matters symmetrically. Our binaries evict pause-invalidated spends when the record's block connects, and `feature_supervised_assets.py` asserts that behaviour (re-run green today on current master). A mempool still holding them a day later is behaviour neither of our two binaries produces, so the version and commit of the node at 13.140.162.77 would tell us whether some build has a real eviction gap, which we would then want fixed, or whether this is a restart and `mempool.dat` subtlety.

The high-fee probe you broadcast is the one item we could not classify from here, because its rejection predates the logging. Send its txid, or simply re-broadcast it now: the explorer will either include it or log exactly why not.

The flag day: move it one day, with your own probe as the gate

Holding 101,200 while the incident was open was the right call, and the incident is now closed: no producer, template, or consensus component failed, and the chain's ability to include transactions is demonstrated above at every entry point, on the record. An indefinite hold from here couples a scheduled consensus change to an incident that is over.

The counterproposal: **the budget fork activates about 24 hours out, at a height derived at merge time as `tip + 1440` rounded up**, so the runway is a true day rather than a number that decays while the branch waits. That gives you the observation window your letter asked for, on a chain you can watch processing transactions.

And the gate is yours rather than ours: **before activation, broadcast a fresh probe from your node, funded from confirmed coins, fee in tSEQ**. If it fails to confirm within three blocks, we hold the fork and debug together with `mempoolrej` on. If it confirms, the network has passed exactly the test your letter said it could not, and the fork proceeds. The cutover will be coordinated with Alberto as usual: it takes the committee from 24.1.0 to 24.3.0 in one step, all nodes at once, and the window will be announced before it starts.